


**PONTIFÍCIA UNIVERSIDADE CATÓLICA DE MINAS GERAIS**

Curso de Engenharia de Computação
Disciplina: Tópicos III — Empreendedorismo e Inovação




# PetCuida

## Documentação final do projeto

Prevenção, créditos solidários e rede local de cuidado animal

**Integrantes:** Gabriel da Silva Cassino
Giovanna Naves Ribeiro
Júlia Rodrigues Vasconcellos Melo
Kathleen Rodrigues Leite Barbosa

**Professor-orientador:** João Carlos Oliveira Caetano
**Professor-tutor:** Paulo Mattos

Belo Horizonte
2026

**Resumo**

O PetCuida é uma plataforma mobile concebida para ampliar o acesso a cuidados veterinários preventivos e de menor custo. A solução reúne calendário de vacinação e vermifugação, prontuário digital, triagem orientativa não diagnóstica, rede local de prestadores e um mecanismo de créditos solidários. Este documento consolida a identificação e a validação do problema, a proposta de valor, a segmentação de clientes, o Business Model Canvas, o protótipo funcional, as estratégias de crescimento e escalabilidade e o plano de próximos passos. As evidências disponíveis indicam aderência inicial da solução às dores relatadas por tutores, mas permanecem exploratórias: os testes foram guiados, a amostra é pequena e ainda não há comprovação de receita, retenção ou disposição a pagar.

**Palavras-chave:** saúde animal; inovação social; plataforma bilateral; prevenção; créditos solidários; prototipagem.

**Abstract**

PetCuida is a mobile platform designed to expand access to preventive and lower-cost veterinary care. The solution integrates a vaccination and deworming schedule, digital medical records, non-diagnostic preliminary screening, a local network of service providers, and a "solidarity credits" mechanism. This document outlines the identification and validation of the problem, the value proposition, customer segmentation, the Business Model Canvas, a functional prototype, growth and scalability strategies, and a roadmap for next steps. Available evidence suggests an initial alignment between the solution and the pain points reported by pet owners; however, these findings remain exploratory—testing was conducted under controlled conditions, the sample size is small, and there is not yet any evidence regarding revenue, user retention, or willingness to pay.

**Keywords:** animal health; social innovation; two-sided platform; prevention; solidarity credits; prototyping.

# Sumário

**1 Visão geral do projeto** **4**

**2 Problema identificado e validação** **4**
2.1 Dor de mercado . . . 4
2.2 Entrevistas exploratórias . . . 4
2.3 Testes de usabilidade e evidências . . . 5
2.4 Limitações da validação . . . 6

**3 Proposta de valor e segmentação** **6**
3.1 Proposta de valor . . . 6
3.2 Diferenciais e salvaguardas . . . 7
3.3 Segmentos prioritários . . . 7

**4 Business Model Canvas** **7**

**5 Protótipo e arquitetura da solução** **8**
5.1 Escopo funcional . . . 8
5.2 Arquitetura técnica registrada . . . 10
5.3 Limitações técnicas atuais . . . 11
5.4 Aplicação web e API em Flask . . . 11

**6 Estratégias de crescimento e escalabilidade** **12**
6.1 Diagnóstico do estágio . . . 12
6.2 Estratégia 1 — Densidade local por parcerias . . . 13
6.3 Estratégia 2 — Aumentar contribuição por usuário . . . 13
6.4 Piloto assistido de oito semanas . . . 13

**7 Plano de próximos passos** **13**
7.1 Equipe e governança . . . 14

**8 Riscos, conformidade e limites** **15**

**9 Conclusão** **16**

**A Links e materiais de referência** **16**

**B Fontes documentais utilizadas** **16**

# 1 Visão geral do projeto

O PetCuida nasce da combinação de duas dificuldades recorrentes: o custo dos serviços veterinários e a ausência de ferramentas simples para organizar o cuidado preventivo. A proposta é conectar tutores, voluntários, organizações comunitárias e prestadores veterinários em uma rede local, na qual o cuidado possa ser planejado, acompanhado e parcialmente viabilizado por créditos obtidos em atividades solidárias.

A solução não pretende substituir consultas ou diagnósticos veterinários. A triagem é orientativa e deve encaminhar situações de risco para atendimento profissional. O produto encontra-se em estágio de validação inicial, com protótipo funcional em Flutter e evidências qualitativas de testes de usabilidade.

Tabela 1: Resumo executivo do PetCuida.

| | |
|---|---|
| **Problema** | Tutores de baixa renda adiam cuidados, recorrem a fontes informais e perdem o histórico do animal. |
| **Solução** | Aplicativo de prevenção, orientação inicial, prontuário, rede solidária e encaminhamento local. |
| **Público prioritário** | Tutores das classes C, D e E, protetores/ONGs e prestadores veterinários regionais. |
| **Diferencial** | Combinação de prevenção, triagem não diagnóstica, créditos solidários, preço social e histórico digital. |
| **Estágio** | Protótipo funcional e validação exploratória; sem receita ou retenção comprovadas. |

# 2 Problema identificado e validação

## 2.1 Dor de mercado

O cuidado com animais de estimação está presente em muitos lares, mas consultas, exames, vacinação e castração podem representar uma barreira severa para famílias com orçamento limitado. As entrevistas realizadas pelo grupo apontaram que o atendimento profissional frequentemente é tratado como último recurso. A prevenção é adiada, o calendário vacinal é esquecido e, diante de sintomas, alguns tutores procuram a internet, administram medicamentos por conta própria ou simplesmente aguardam uma melhora espontânea.

A dor também é operacional: a troca de profissionais e a ausência de um registro centralizado fazem com que o histórico médico do animal se perca. Em emergências, somam-se a distância até uma unidade disponível e a impossibilidade de comparar preços com calma. O resultado é um ciclo de atraso, agravamento do problema e aumento do custo.

## 2.2 Entrevistas exploratórias

Foram relatados oito depoimentos de tutores com perfis distintos. Os padrões mais relevantes foram:

- consultas e exames são percebidos como caros, sobretudo quando a situação já se tornou emergencial;
- vacinação e acompanhamento preventivo são irregulares ou dependem de campanhas públicas;
- há insegurança para decidir quando procurar atendimento e tendência a buscar orientação informal;
- animais resgatados e tutores de baixa renda têm forte vínculo afetivo, mas pouca capacidade de pagamento;
- a falta de atendimento 24 horas próximo agrava a vulnerabilidade em situações urgentes;
- há interesse em uma rede acessível, desde que exista confiança, orientação e divulgação local.

Uma formulação representativa da necessidade é: *cuidar do animal com dignidade, mesmo quando o tutor não consegue pagar o preço integral de um atendimento particular.* A partir disso, o problema não é apenas localizar um prestador: envolve prevenção, orientação confiável, acesso financeiro e continuidade do cuidado.

## 2.3 Testes de usabilidade e evidências

O protótipo atual, denominado D4 (revisão interna `PetCuida_D4C2B_LGPD`), foi desenvolvido e disponibilizado ao longo de cinco revisões. Nas duas primeiras etapas, realizaram-se entrevistas para apresentar a proposta inicial, acompanhadas da transcrição integral dos diálogos e da identificação dos participantes, a fim de preparar as fases seguintes. Nas três revisões subsequentes, registraram-se apenas anotações dos pontos considerados relevantes, sem a identificação demográfica completa. Como a navegação foi conduzida pelos entrevistadores, o procedimento deve ser interpretado como uma demonstração guiada com coleta de feedback, e não como um teste autônomo de tarefas da aplicação.

Os resultados qualitativos obtidos foram majoritariamente favoráveis. A participante Aline comparou a experiência à de um aplicativo de mensagens e considerou a interface intuitiva. Já Galdina e Atanizia avaliaram a solução como interessante, mas destacaram a necessidade de maior divulgação. Outros relatos valorizaram funcionalidades específicas, como a triagem, os lembretes, a área dedicada ao pet, o prontuário e a centralização de diversas funções em um único ambiente.

Ressalta-se que a última versão conta com uma interface voltada para pessoa jurídica, desenvolvida ainda em estágio inicial e com margem para aprimoramentos. Adicionalmente, foi disponibilizado um formulário no Google Forms para a realização de uma pesquisa de mercado.

Tabela 2: Principais achados da validação exploratória.

| Achado | Interpretação | Situação |
|---|---|---|
| Facilidade percebida | Navegação principal considerada tranquila e familiar. | Confirmado qualitativamente |
| Triagem orientativa | Funcionalidade percebida como prática para decidir o próximo passo. | Manter e validar com supervisão técnica |
| Créditos solidários | Ideia compreendida parcialmente; faltou clareza sobre geração e uso. | Prioridade de melhoria |
| QR Code do prontuário | Finalidade não foi compreendida por uma participante. | Adicionar explicação contextual |
| Mapa e login | Ainda esquemáticos/simulados na versão alfa. | Integrações futuras |

### 2.4 Limitações da validação

A amostra analisada é reduzida, não probabilística e parcialmente anônima. Em razão do baixo volume de dados coletados, não foram mensuradas métricas quantitativas de desempenho e negócio, como taxa de sucesso, tempo de execução de tarefas, frequência de erros, retenção, conversão ou disposição real de pagamento. Além disso, a coleta inicial ocorreu em um contexto informal, estando sujeita ao viés de cortesia devido à proximidade entre entrevistadores e participantes. Dessa forma, embora os resultados sustentem hipóteses iniciais de valor e usabilidade, eles não permitem generalizações para todo o público de tutores nem comprovam tração de mercado.

## 3 Proposta de valor e segmentação

### 3.1 Proposta de valor

A proposta central do projeto é: **"Cuidar antes, pagar de forma possível e encontrar orientação confiável: o PetCuida conecta tutores, serviços comunitários e prestadores veterinários em uma rede local de prevenção e cuidado acessível."**

Para o tutor, o principal benefício é transformar o cuidado com o animal em uma rotina organizada, apoiada por lembretes, prontuário digital e orientação inicial. Quando houver necessidade de atendimento, a rede permite localizar parceiros, consultar preços sociais e combinar créditos da plataforma com pagamento monetário. Além disso, possibilita o incremento de renda por meio da prestação de serviços, caso o usuário deseje e esteja apto a realizá-los.

Para clínicas, veterinários e profissionais liberais, a plataforma atua como um canal para gerar demanda recorrente, ocupar horários ociosos e ampliar o alcance social de seus atendimentos. Por fim, para ONGs, instituições de ensino superior, protetores independentes e entidades similares, a solução potencializa o impacto social ao facilitar a oferta de atendimentos e serviços à comunidade.

## 3.2 Diferenciais e salvaguardas

1. **Prevenção ativa:** calendário de vacinação e vermifugação com lembretes.
2. **Triagem não diagnóstica:** árvore orientativa para classificar urgência, agendamento ou observação, com encaminhamento profissional para sinais de risco.
3. **Créditos solidários:** conversão de atividades comunitárias em saldo utilizável em serviços parceiros.
4. **Prontuário digital:** histórico centralizado e exportável, evitando perda de informação.
5. **Rede local:** curadoria de prestadores, preços sociais, reputação e campanhas comunitárias.

A confiança depende de curadoria de parceiros, transparência de preços, regras auditáveis para os créditos, proteção de dados e linguagem explícita de que a triagem não substitui um médico-veterinário.

## 3.3 Segmentos prioritários

Tabela 3: Segmentos, dores e valor entregue.

| Segmento | Dores principais | Valor do PetCuida |
|---|---|---|
| Tutores de baixa renda | Custo, medo de emergências caras, falta de informação e prevenção. | Preços sociais, créditos, agendamento, lembretes e prontuário. |
| Tutores voluntários | Tempo disponível, mas pouco dinheiro para cuidar do próprio pet. | Troca de serviços por créditos veterinários. |
| ONGs e protetores | Muitos animais, recursos escassos e registros dispersos. | Rede, campanhas, voluntários e histórico centralizado. |
| Clínicas e veterinários | Agenda ociosa e baixa retenção local. | Novos clientes, ocupação e reputação de impacto social. |

# 4 Business Model Canvas

O PetCuida opera como uma plataforma bilateral com camada solidária. O foco inicial deve ser hiperlocal, pois a troca só cria valor quando há densidade suficiente de tutores, voluntários e prestadores no mesmo território.

Tabela 4: Business Model Canvas do PetCuida.

| **Bloco** | **Definição do modelo** |
|---|---|
| Parcerias principais | Clínicas e veterinários regionais; ONGs e protetores; UBS Animal e secretarias; faculdades de Medicina Veterinária; pet shops, farmácias e laboratórios; fornecedores de nuvem e pagamentos. |
| Atividades-chave | Desenvolver e operar o app; credenciar prestadores; gerir créditos; manter protocolos; moderar a comunidade; educar preventivamente; proteger dados. |
| Proposta de valor | Prevenção, triagem orientativa, créditos solidários, preços transparentes, pagamento misto, prontuário único e ocupação de horários ociosos. |
| Relacionamento | Onboarding guiado; notificações; comunidade; suporte humano; avaliações mútuas; campanhas de vacinação e castração. |
| Segmentos | Tutores de baixa renda; tutores com tempo disponível; ONGs/protetores; clínicas e veterinários; pagadores institucionais. |
| Recursos-chave | App Flutter; Firebase; carteira de créditos; protocolos veterinários; base de prontuários; rede credenciada; equipe enxuta; marca comunitária. |
| Canais | Google Play; indicação em ONGs e grupos locais; QR codes em parceiros; eventos; redes sociais e conteúdo preventivo. |
| Estrutura de custos | Desenvolvimento; nuvem; consultoria veterinária; credenciamento; suporte/moderação; prevenção de fraude; marketing; pagamentos; LGPD. |
| Fontes de receita | Comissão de 8%–12% sobre dinheiro intermediado; PetCuida+; planos para clínicas; convênios e patrocínios; comissão de produtos parceiros. A troca de tempo permanece gratuita. |

A coerência do modelo está em monetizar os fluxos pagos, os serviços às clínicas e os convênios, preservando gratuitamente o núcleo de prevenção e a utilização dos créditos pelo público de baixa renda. Os principais riscos são liquidez local, adesão de prestadores, fraude, responsabilidade técnica e sustentabilidade do fundo de créditos.

# 5 Protótipo e arquitetura da solução

## 5.1 Escopo funcional

O protótipo desenvolvido em Flutter contempla as seguintes funcionalidades: autenticação, *onboarding*, painel inicial, cadastro do pet, controle de vacinas, calendário, prontuário, geração de QR Code, triagem via questionário ou chat, rede solidária, mapa esquemático, agendamento, mural, perfil, gestão de saldo e suporte. Na versão alfa, os dados são armazenados localmente de forma simulada (*mock*), e algumas integrações externas ainda operam em ambiente de simulação. Na revisão atual, incorporou-se um painel exclusivo

para pessoas jurídicas, permitindo também o cadastro de seus respectivos funcionários.




Figura 1: Painel inicial do protótipo PetCuida.

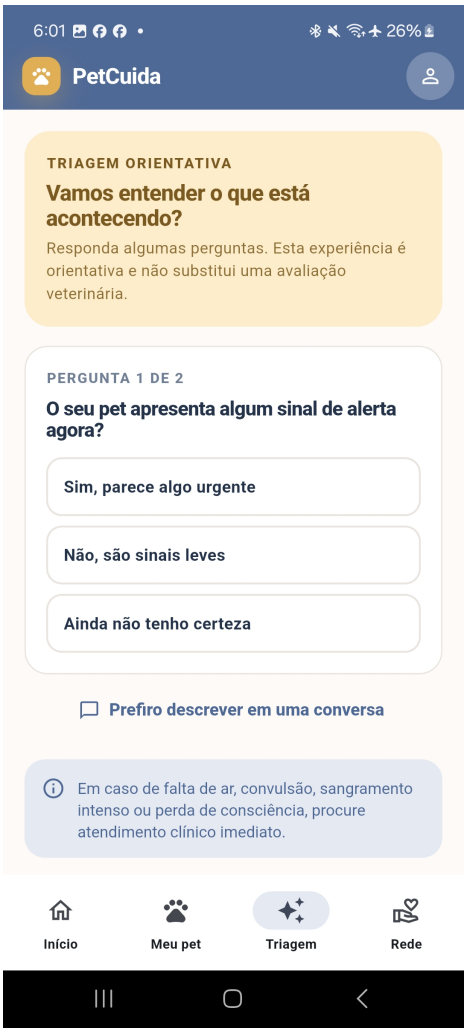



Figura 2: Fluxo de triagem orientativa.

## 5.2 Arquitetura técnica registrada

O projeto organiza o código por funcionalidades, com núcleo compartilhado:

```
lib/
  main.dart
  core/       theme, models, services, widgets, routes
  features/   auth, home, pet, triagem,
              rede_solidaria, perfil
```

As decisões registradas incluem `provider` com `ChangeNotifier` para estado, `go_router` para navegação e redirecionamento, `qr_flutter` para o prontuário e fontes do sistema para manter o build leve. A compatibilidade declarada é Android 10 ou superior, com `minSdk 29` e `targetSdk 35`.




Figura 3: Área do pet com dados e histórico.

## 5.3 Limitações técnicas atuais

O mapa é esquemático; créditos, registros e login Google são demonstrativos; ainda faltam backend produtivo, pagamentos reais, notificações push, GPS, compartilhamento externo e controles completos de segurança. A triagem não realiza diagnóstico e deve permanecer subordinada a protocolos revisados por profissional habilitado.

## 5.4 Aplicação web e API em Flask

Além do aplicativo Flutter e da prévia estática, o pacote contempla uma implementação web completa em Python utilizando o microframework Flask, localizada no diretório `flask/`. A aplicação adota uma arquitetura desacoplada: o backend expõe rotas de API RESTful (definidas em `routes.py`), enquanto a interface do usuário é renderizada como uma *Single Page Application* (SPA) reativa por meio de Vue.js (`vue.esm-browser.prod.js`) consumindo o cliente HTTP em `api.js`.

A estrutura funcional da versão Flask espelha e expande os fluxos do protótipo mobile, oferecendo autenticação, painel do tutor (`Dashboard.js`) e painel administrativo para pessoas jurídicas (`OrganizationDashboard.js`). Um diferencial relevante dessa implementação é a camada de abstração de persistência (`repository.py` e `data_backends/`), que permite alternar entre armazenamento local ou bancos em nuvem por meio de adaptadores dedicados para Firebase Firestore e Supabase, além de incorporar controles explícitos de conformidade e anonimização previstos na Lei Geral de Proteção de Dados (`privacy.py` e `PRIVACIDADE_LGPD.md`).

Tabela 5: Principais módulos e arquivos da aplicação Flask recebidos no pacote.

| **Arquivo / Diretório** | **Função** |
|---|---|
| `run.py` e `app/__init__.py` | Ponto de entrada da aplicação e inicialização da instância Flask. |
| `app/routes.py` | Definição dos *endpoints* da API REST e renderização da página base. |
| `app/repository.py` e `db.py` | Camada de acesso a dados e gerenciamento de conexões. |
| `app/data_backends/` | Adaptadores de persistência para Firestore e Supabase. |
| `app/privacy.py` | Regras de tratamento, consentimento e conformidade com a LGPD. |
| `app/static/js/components/` | Componentes Vue.js (`Login`, `Dashboard` e `OrganizationDashboard`). |
| `tests/` | Suíte de testes automatizados de rotas (`test_app.py`) e privacidade (`test_privacy.py`). |

Para implantação em ambiente de produção, recomenda-se substituir o servidor de desenvolvimento do Flask por um servidor WSGI robusto (como Gunicorn ou uWSGI), configurar rigorosamente as variáveis de ambiente descritas em `.env.example`, ativar os adaptadores de banco de dados em nuvem com políticas de controle de acesso por perfil (PF/PJ) e executar continuamente a suíte de testes em `tests/` para assegurar a integridade das regras da LGPD.

# 6 Estratégias de crescimento e escalabilidade

## 6.1 Diagnóstico do estágio

A PetCuida está em validação inicial, antes da tração. Há protótipo e evidência exploratória de usabilidade, mas não há nos materiais analisados usuários pagantes, receita recorrente contratada, retenção medida ou contratos comerciais ativos. O setor pet é amplo, mas esse dado não substitui a validação de uma praça local.

## 6.2 Estratégia 1 — Densidade local por parcerias

A primeira estratégia é concentrar a operação em uma praça de Belo Horizonte, com duas ou três clínicas e até duas organizações comunitárias. A aquisição deve ocorrer por co-marketing, QR codes individualizados, indicação e eventos. O indicador não será download, mas cadastro ativado, encaminhamento concluído, retenção e oferta realmente disponível.

## 6.3 Estratégia 2 — Aumentar contribuição por usuário

A segunda estratégia testa a disposição a pagar por PetCuida+ e Plano Parceiro, além de patrocínios formais de campanhas. Esses serviços só devem ser ampliados após comprovação de demanda, conversão e margem. Intenção declarada não deve ser confundida com pagamento efetivo.

## 6.4 Piloto assistido de oito semanas

Tabela 6: Plano do piloto territorial.

| Semanas | Foco | Entregas e evidências |
|---|---|---|
| 1–2 | Descoberta | 15–20 entrevistas; conversas com 5 clínicas e 2 organizações. |
| 3–4 | Usabilidade | 10 testes autônomos, sem ajuda; taxa de conclusão, erros e dúvidas. |
| 5–7 | Piloto concierge | Até 40–50 tutores e 2–3 clínicas, com acompanhamento manual e salvaguardas. |
| 8 | Decisão | Continuar, iterar ou interromper com base em contagens, taxas e limitações. |

Para escalar, a equipe deve primeiro padronizar processos e depois automatizá-los: onboarding de clínicas, QR codes, lembretes, indicadores, suporte e conciliação financeira. O "kit de praça" deve conter checklist, contrato revisado, critérios de credenciamento, materiais, treinamento, política de créditos e painel.

# 7 Plano de próximos passos

Para evoluir o protótipo atual (`D4C2B_LGPD`) rumo a um piloto validado em ambiente real, estabeleceu-se um plano de ação dividido em frentes de usabilidade, engenharia, conformidade e validação de mercado:

1. **Refinamento de interface e fluxos:** Aprimorar a comunicação visual do sistema de créditos híbridos, inserir orientações contextuais sobre o uso do QR Code na própria tela e consolidar o painel para pessoas jurídicas (*Organization Dashboard*).

2. **Avaliação de usabilidade autônoma:** Conduzir uma nova rodada de testes sem mediação dos entrevistadores, mensurando taxas de conclusão e erros, além de aplicar um instrumento padronizado, como a escala SUS (*System Usability Scale*).

3. **Validação clínica e ética:** Revisar os fluxos de triagem orientativa junto a médicos-veterinários parceiros, garantindo a manutenção explícita de avisos de que a ferramenta não substitui o diagnóstico clínico presencial.

4. **Governança e conformidade (LGPD):** Concluir a implementação dos requisitos previstos na documentação de privacidade do projeto, abrangendo gestão de consentimento, minimização de dados, controle de acesso por perfil (PF/PJ), políticas de retenção, rotinas de *backup*, trilhas de auditoria (*logs*) e plano de resposta a incidentes.

5. **Integração de persistência em nuvem:** Substituir os dados locais simulados (*mock*) pela ativação dos adaptadores de banco de dados em nuvem (Firestore ou Supabase) já estruturados na arquitetura do backend.

6. **Operação piloto local:** Credenciar formalmente a primeira praça de operação, iniciando com duas a três clínicas veterinárias e organizações comunitárias (ONGs ou projetos universitários) para validar o fluxo real de atendimento.

7. **Instrumentação de métricas:** Implementar a coleta de indicadores de produto e negócio, incluindo ativação (D7), retenção (D30), custo de aquisição de clientes (CAC) por canal, taxa de conversão paga, margem por atendimento e sustentabilidade do fundo de créditos.

8. **Validação de monetização:** Testar a disposição real de pagamento por meio de pré-adesões, contratos piloto com parceiros ou transações reais, mantendo a distinção rigorosa entre projeções financeiras e resultados empíricos.

9. **Documentação e governança da equipe:** Consolidar o manual de implantação em produção e revisar periodicamente a matriz de responsabilidades entre os integrantes a cada ciclo de desenvolvimento.

## 7.1 Equipe e governança

A estrutura organizacional proposta busca ser funcional e enxuta, distribuindo as responsabilidades entre os integrantes da seguinte forma: Giovanna coordena a direção geral e a sustentabilidade financeira; Gabriel e Kathleen conduzem o desenvolvimento de produto e tecnologia; Kathleen também acompanha as operações e parcerias locais; e Júlia lidera as frentes de marketing, relacionamento, experiência do usuário e suporte. A governança operacional prevê reuniões semanais de acompanhamento, revisões quinzenais de produto, avaliações mensais de negócio e o registro contínuo das decisões tomadas.

Para orientar essas revisões e sustentar a tomada de decisão baseada em evidências, definiram-se os indicadores operacionais apresentados na Tabela 7.

Tabela 7: Indicadores e gatilhos de decisão.

| **Indicador** | **Definição operacional** |
|---|---|
| Ativação D7 | Perfil do animal completo e ao menos um lembrete cadastrado em até sete dias. |
| Retenção D30 | Realização de ao menos uma ação de valor na plataforma entre os dias 22 e 30. |
| CAC ativado | Gastos atribuíveis ao canal divididos pelo total de usuários que atingiram a ativação D7. |
| Conversão paga | Assinantes pagantes ativos divididos pelos usuários ativos elegíveis, excluindo períodos de teste gratuito. |
| Margem por atendimento | Comissão recebida subtraída de taxas financeiras, impostos, custos variáveis e créditos liquidados. |

A abertura de uma segunda praça de operação deve depender do cumprimento rigoroso dos seguintes critérios: dois ciclos mensais consecutivos com oferta contratada, coortes mensuráveis, CAC compatível com o retorno (*payback*/CLV), margem unitária não negativa (ou subsídio formalmente contratado), receita efetivamente recebida e estabilidade operacional.

# 8 Riscos, conformidade e limites

A operação do PetCuida envolve desafios clínicos, financeiros, operacionais e regulatórios que exigem salvaguardas explícitas. A Tabela 8 sintetiza os riscos prioritários mapeados e as respectivas estratégias de mitigação.

Tabela 8: Matriz de riscos prioritários e ações de mitigação.

| **Risco mapeado** | **Resposta e salvaguarda prevista** |
|---|---|
| Interpretação da triagem como diagnóstico | Revisão de fluxos por médicos-veterinários e exibição permanente de avisos de caráter estritamente orientativo. |
| Fraude ou insuficiência do fundo de créditos | Regras auditáveis de emissão, limites de uso por usuário, lastro financeiro e conciliação periódica. |
| Baixa liquidez local | Foco hiperlocal na primeira praça e curadoria ativa de clínicas e organizações parceiras. |
| Privacidade e segurança da informação | Gestão de consentimento informado, minimização de dados, controle de acesso por perfil (PF/PJ) e trilhas de auditoria (*logs*). |
| Atendimento manual excessivo | Padronização de processos via “kit de praça” antes da automação gradual de fluxos. |
| Baixa disposição a pagar e métricas infladas | Validação com transações reais e monitoramento de ativação (D7) e retenção (D30) em vez de apenas *downloads*. |

Em relação à conformidade legal, os dados pessoais dos tutores, os registros cadastrais de parceiros e o histórico clínico dos animais devem ser tratados em estrita observância à Lei Geral de Proteção de Dados (LGPD) e às diretrizes da Autoridade Nacional de Proteção de Dados (ANPD). Ressalta-se que, embora a arquitetura atual já contemple módulos iniciais de privacidade, a implantação produtiva integral dessas salvaguardas constitui requisito obrigatório antes da operação com usuários reais.

# 9 Conclusão

O PetCuida apresenta-se como uma resposta estruturada e coerente para uma dor validada qualitativamente: tutores de baixa renda necessitam de prevenção acessível, orientação inicial confiável e mecanismos financeiros viáveis, e não apenas de um catálogo de clínicas. O ecossistema prototipado — composto pelo aplicativo mobile em Flutter, pela aplicação web e API em Flask e pelos módulos iniciais de conformidade com a LGPD — integra funcionalidades reconhecidas como úteis pelos participantes, com destaque para a triagem orientativa, os lembretes preventivos, o prontuário digital e a rede solidária.

A principal conclusão estratégica deste ciclo é que a equipe deve comprovar a densidade local e a economia unitária antes de buscar escala ampla. O próximo passo prioritário não consiste em maximizar o volume de *downloads*, mas em executar o piloto assistido de oito semanas, mensurar o comportamento real dos usuários, validar a disposição efetiva de pagamento, aprimorar a comunicação do sistema de créditos e consolidar uma rede confiável de oferta. Dessa forma, a expansão do projeto ocorrerá orientada por evidências empíricas, responsabilidade técnica e rigor na proteção de dados.

O grupo mantém a visão de que a prevenção, aliada à economia solidária, é um caminho promissor para democratizar o acesso à saúde veterinária no Brasil.

# A Links e materiais de referência

- Repositório do projeto: https://github.com/kasshinokun/Q3_Q4_2026_Public/tree/main/TOPICOS_III/
- Pasta do protótipo e código: https://github.com/kasshinokun/Q3_Q4_2026_Public/tree/main/TOPICOS_III/Atividade_VI
- Aplicação web temporária indicada no README: https://petcuida.pythonanywhere.com
- Registro em vídeo dos testes: https://youtu.be/uS83wGpVHdg

# B Fontes documentais utilizadas

1. Atividade I-b — Identificação da dor de mercado e relatos de oito entrevistas, 2026.
2. Atividade III — Business Model Canvas PetCuida, 2026.

3. Atividade IV — Proposta de valor, segmentação de clientes e arquitetura de negócio, 2026.

4. Atividade VI — Relatório de testes de usabilidade do aplicativo PetCuida Alpha 3i, 2026.

5. Atividade XI — Estrutura organizacional e responsabilidades, 2026.

6. Atividade XIII — Estratégias de crescimento, escalabilidade e viabilidade, 2026.

7. Manual de Entrega 14 do Projeto — Documento final da disciplina, 2026.




**PONTIFÍCIA UNIVERSIDADE CATÓLICA DE MINAS GERAIS**
Campus Coração Eucarístico
Curso de Engenharia de Computação

Disciplina: Tópicos III – Empreendedorismo e Inovação

# Documentação Técnica de Software:

# PetCuida Android

Arquitetura, Configuração e Requisitos de Evolução

**Integrantes:**

Gabriel da Silva Cassino
Giovanna Naves Ribeiro
Júlia Rodrigues Vasconcellos Melo
Kathleen Rodrigues Leite Barbosa

**Professor-orientador:**
João Carlos Oliveira Caetano
**Professor-tutor:**
Paulo Mattos

Belo Horizonte
2026

**Gabriel da Silva Cassino**
**Giovanna Naves Ribeiro**
**Júlia Rodrigues Vasconcellos Melo**
**Kathleen Rodrigues Leite Barbosa**

# Documentação Técnica de Software: PetCuida Android

(Especificação da Aplicação Mobile — Revisão `D4C2B_LGPD`)

Documentação técnica apresentada à disciplina Tópicos III – Empreendedorismo e Inovação, do Curso de Engenharia de Computação da PUC Minas, como artefato complementar de engenharia referente à arquitetura mobile do projeto PetCuida.

Belo Horizonte
2026

# Sumário

**1 Identificação e arquitetura** **2**

**2 Configuração Android observada** **2**

**3 Fluxos e tratamento de dados** **2**

**4 Execução e validação** **3**

**5 Segurança e requisitos de evolução** **3**

**6 Arquivos relevantes** **4**

**Referências** **4**

# 1 Identificação e arquitetura

O **PetCuida Android** constitui o destino nativo para dispositivos Android desenvolvido a partir da mesma base de código Flutter utilizada na versão Web. O código-fonte em Dart reside no diretório `lib/`, enquanto as configurações específicas da plataforma nativa encontram-se em `android/`. O identificador de pacote declarado no Gradle é `com.grupo4topicos3.petcuida.petcuida`, utilizando o rótulo de exibição `PetCuida` e a versão `1.3.0+1` definida no arquivo `pubspec.yaml`.

A camada nativa inicializa a classe `MainActivity` utilizando o padrão *Flutter embedding v2*. As rotas, o gerenciamento de estado e os componentes visuais (*widgets*) são integralmente compartilhados com a aplicação Web. Na entrega atual, a camada de dados opera por meio de uma simulação em memória volátil (*mock*), e a interface informa explicitamente ao usuário que a autenticação e as integrações externas são demonstrativas.

# 2 Configuração Android observada

O arquivo `android/app/build.gradle.kts` herda dinamicamente as propriedades `compileSdk`, `targetSdk`, `minSdk`, versionamento e NDK diretamente da configuração do Flutter, fazendo com que os valores efetivos dependam do SDK instalado no ambiente de compilação. A compatibilidade de compilação para Java e Kotlin está configurada para a **JVM 17**.

O manifesto Android (`AndroidManifest.xml`) define o rótulo `PetCuida`, configura a *activity* principal como exportada para o inicializador do sistema (*launcher*) e declara a permissão de acesso à câmera. O recurso de hardware de câmera foi marcado como opcional (`required="false"`) para assegurar que o aplicativo possa ser instalado em dispositivos que não possuam câmera integrada.

Ressalta-se que o bloco de compilação `release` no Gradle está atualmente configurado para assinar o pacote com a chave padrão de depuração (*debug*). Essa configuração atende apenas a testes internos e não constitui um perfil distribuível em lojas oficiais. Adicionalmente, o pacote de entrega omite o arquivo `local.properties` (que armazena caminhos locais da máquina de desenvolvimento) e não inclui credenciais produtivas do Firebase.

# 3 Fluxos e tratamento de dados

A experiência do protótipo contempla as telas de abertura (*splash*), autenticação e cadastro simulados, *onboarding*, painel principal, perfil do tutor, gestão de pets, triagem orientativa, rede solidária, agendamentos e painel corporativo para organizações parceiras. Tanto o cadastro quanto a sessão ativa não são persistidos em banco de dados nem vinculados a um provedor de identidade real. Todas as ações executadas alteram apenas o estado em memória, sendo reinicializadas ao encerrar o processo do aplicativo.

A permissão de câmera atende exclusivamente ao fluxo de seleção de imagem para o perfil do animal, sem realizar envio (*upload*) para servidores remotos. Não há sincronização em nuvem de prontuários, processamento de pagamentos, saldo financeiro real, envio de notificações *push* ou integração ativa com clínicas. Por fim, o módulo de triagem atua estritamente como guia orientativo de interface, não possuindo caráter de diagnóstico veterinário.

# 4 Execução e validação

Para compilar, testar e executar o projeto em um ambiente configurado com Flutter SDK, Android SDK, Java 17 e um dispositivo físico ou emulador, devem-se executar os seguintes comandos na raiz do projeto Flutter:

- `flutter pub get` — instala e resolve as dependências declaradas;
- `flutter analyze` — executa a análise estática de código e verificação de tipos;
- `flutter test` — executa a suíte de testes unitários e de *widgets*;
- `flutter run -d <id-do-dispositivo>` — inicia a aplicação em modo de depuração;
- `flutter build apk --debug` — gera o pacote instalável (APK) para testes internos.

Antes de gerar uma versão de produção (*release*), é indispensável configurar uma chave de assinatura segura e verificar o versionamento, as versões efetivas do SDK, as permissões declaradas e a árvore de dependências. Nota-se que o *Gradle wrapper* isoladamente não compila o projeto sem a presença do SDK do Flutter no ambiente.

# 5 Segurança e requisitos de evolução

O APK de demonstração não deve ser utilizado com dados pessoais reais nem implantado como serviço ativo. Para a transição rumo ao piloto controlado, estabelecem-se os seguintes requisitos técnicos obrigatórios:

1. **Autenticação e controle de acesso:** Implementar autenticação real com sessão persistente, derivar a propriedade dos registros a partir de identidade confiável, testar o isolamento entre contas e aplicar políticas de segurança do tipo *default-deny* no banco de dados.
2. **Saneamento de ambiente:** Remover dados iniciais de demonstração (*seeds*) dos ambientes remotos e migrar o modelo de dados para a camada produtiva.
3. **Conformidade com a LGPD:** Implementar mecanismos formais de gestão de consentimento, controle de armazenamento de imagens, políticas de retenção e exclusão de dados, além de exibir mensagens claras de falha de rede sem simular falso sucesso.
4. **Distribuição e hardware:** Configurar assinatura criptográfica de *release* protegida fora do repositório de código e reavaliar a real necessidade da permissão de câmera, mantendo-a opcional.
5. **Qualidade e resiliência:** Validar o comportamento de formulários, teclado virtual, escalonamento de fontes, rotação de tela, ciclo de vida da aplicação (retorno após suspensão) e recuperação de conectividade em dispositivos reais, além de executar testes de autorização no backend antes de inserir credenciais de produção.

# 6 Arquivos relevantes

A Tabela 1 relaciona os principais arquivos estruturais que compõem a implementação Android e a camada compartilhada do aplicativo.

Tabela 1: Principais arquivos da implementação Android e da camada compartilhada.

| **Arquivo** | **Função** |
|---|---|
| `android/app/src/main/AndroidManifest.xml` | Define o rótulo, o ponto de entrada (*launcher*) e as permissões do sistema. |
| `android/app/build.gradle.kts` | Especifica a identidade do pacote e as opções de compilação Android. |
| `android/app/src/main/kotlin/../MainActivity.kt` | Implementa a *activity* nativa hospedeira do Flutter. |
| `lib/main.dart` | Ponto de entrada e raiz de composição compartilhada da aplicação. |
| `lib/core/services/app_data_service.dart` | Gerencia o estado e os dados simulados da demonstração. |
| `lib/core/services/auth_service.dart` | Implementa o fluxo de autenticação demonstrativa sem backend. |
| `test/core/services/app_data_service_test.dart` | Contém os casos de testes unitários da camada de serviços. |

# Referências


[1] GRUPO 4. **Documentação Própria A: código-fonte e configuração Android (`flutter/`)**. Belo Horizonte: PUC Minas, 2026.

[2] GRUPO 4. **Pacote de entrega `PetCuida_D4C2B_LGPD` e Protótipos Iniciais**. Belo Horizonte: PUC Minas, 2026.

[3] GRUPO 4. **Documentação Própria B: proposta técnica e critérios de aceite para Web e Mobile**. Belo Horizonte: PUC Minas, 2026.




**PONTIFÍCIA UNIVERSIDADE CATÓLICA DE MINAS GERAIS**
Campus Coração Eucarístico
Curso de Engenharia de Computação

Disciplina: Tópicos III – Empreendedorismo e Inovação

# Documentação Técnica de Software:

# PetCuida Web e Firebase Hosting

Arquitetura Web Flutter, Prévia Estática e Requisitos de Implantação

**Integrantes:**

Gabriel da Silva Cassino
Giovanna Naves Ribeiro
Júlia Rodrigues Vasconcellos Melo
Kathleen Rodrigues Leite Barbosa

**Professor-orientador:**
João Carlos Oliveira Caetano
**Professor-tutor:**
Paulo Mattos

Belo Horizonte
2026

**Gabriel da Silva Cassino**
**Giovanna Naves Ribeiro**
**Júlia Rodrigues Vasconcellos Melo**
**Kathleen Rodrigues Leite Barbosa**

# Documentação Técnica de Software: PetCuida Web e Firebase Hosting

(Especificação das Interfaces Web — Revisão `D4C2B_LGPD`)

Documentação técnica apresentada à disciplina Tópicos III – Empreendedorismo e Inovação, do Curso de Engenharia de Computação da PUC Minas, como artefato complementar de engenharia referente à arquitetura Web e hospedagem do projeto PetCuida.

Belo Horizonte
2026

# Sumário

1 Identificação e arquitetura 2

2 Funcionalidades observáveis (Flutter Web) 2

3 Metadados e comportamento Web 2

4 Prévia web estática no Firebase Hosting 2

5 Dependências, execução e implantação 3

6 Limites e requisitos de evolução 3

7 Estrutura relevante 4

Referências 4

# 1 Identificação e arquitetura

A frente Web do **PetCuida** contempla duas abordagens complementares no pacote entregue: (i) a compilação **Flutter Web**, que atua como destino para navegadores a partir da mesma base de código compartilhada com o aplicativo Android, e (ii) uma **prévia web estática** configurada especificamente para demonstrações rápidas no **Firebase Hosting**.

Na frente Flutter Web, o código-fonte em Dart reside em `flutter/lib/` e os arquivos de configuração do navegador ficam em `flutter/web/`. O projeto utiliza o sistema visual Material, o pacote `Provider` para gerenciamento de estado e o `GoRouter` para roteamento, organizando modelos e serviços em `core/` e `features/`. O ponto de entrada em `lib/main.dart` configura a localização `pt_BR`, instancia os serviços de demonstração em memória (`AppDataService` e `AuthService`) e exibe um aviso global informando o caráter demonstrativo dos dados.

# 2 Funcionalidades observáveis (Flutter Web)

O roteador do aplicativo Web define fluxos de abertura (*splash*), login, cadastro e *onboarding*, dando acesso às áreas principais de início, gestão do pet, triagem orientativa e rede solidária. Também estão mapeadas rotas para agenda, mural comunitário, perfil, histórico clínico, saldo e extrato de créditos, configurações, suporte e painel corporativo para organizações parceiras.

Na simulação atual, os dados iniciais (*seeds*) representam perfis fictícios, catálogo local de serviços e pedidos demonstrativos. Ações como agendar um atendimento ou candidatar-se a um pedido atualizam o estado apenas na sessão ativa em memória, sendo descartadas ao recarregar a página ou encerrar o processo, sem sincronização entre navegador e dispositivos móveis.

# 3 Metadados e comportamento Web

A configuração atualizada de `web/index.html` e `web/manifest.json` define o idioma HTML `pt-BR`, título e descrição próprios, *viewport*, cor de tema e manifesto PWA (*Progressive Web App*) com a identidade visual do PetCuida, substituindo os textos padrão gerados pelo Flutter.

Permanecem como verificações pendentes os testes de instalação PWA, navegação direta via *deep links*, critérios de acessibilidade, SEO, cache *offline* e compatibilidade entre diferentes navegadores. Além disso, caso a compilação Web seja hospedada em um subdiretório, o atributo `base href` deverá ser ajustado no momento do *build*.

# 4 Prévia web estática no Firebase Hosting

Além da aplicação compilada em Flutter, o pacote inclui uma prévia web estática em HTML/JavaScript puro destinada a demonstrações ágeis via **Firebase Hosting**. A configuração de implantação é controlada pelo arquivo `firebase.json`, que define o diretório público como a própria raiz do pacote (`"public": "."`) com regras de exclusão de arquivos internos, e pelo arquivo `firebaserc`, que vincula o projeto padrão `petcuida-mobile-2026`.

O pacote preserva duas versões da prévia estática: o arquivo `index_0.html`, que apresenta um fluxo inicial simplificado (navegação entre início, pets, rede e agenda), e o arquivo `index_1a.html`, que constitui o ponto de entrada mais completo. O `index_1a.html` simula diretamente no navegador a autenticação para pessoa física (PF) e pessoa jurídica (PJ), alternância de perfis, painel inicial, agenda, rede solidária, área do pet, prontuário e geração dinâmica de QR Code.

Ressalta-se que os dados exibidos nessa prévia são estritamente demonstrativos, não havendo autenticação real, persistência de contas, backend de prontuário ou processamento financeiro. Como a biblioteca de geração de QR Code é carregada via CDN externa, essa funcionalidade exige conexão ativa com a internet para operar.

# 5 Dependências, execução e implantação

No módulo Flutter Web, as dependências são declaradas em `pubspec.yaml`, abrangendo Flutter, Provider, GoRouter, internacionalização, seleção de imagens e adaptadores remotos. Embora os SDKs do Firebase e do Supabase constem no arquivo de dependências, o ponto de entrada atual não os inicializa, mantendo a operação restrita à memória local.

Em um ambiente equipado com Flutter SDK e Google Chrome, os comandos para validação e geração dos artefatos Web são:

- `flutter pub get` — baixa e resolve as dependências do projeto;
- `flutter analyze` — realiza a análise estática do código Dart;
- `flutter test` — executa a suíte de testes unitários;
- `flutter run -d chrome` — executa a aplicação em modo de desenvolvimento no navegador;
- `flutter build web` — compila os artefatos estáticos finais na pasta `build/web/`.

Para a publicação da prévia estática no Firebase Hosting, após instalar o Firebase CLI e autenticar a conta vinculada ao projeto `petcuida-mobile-2026`, utiliza-se o comando `firebase deploy --only hosting`, certificando-se previamente de definir qual arquivo HTML (`index_1a.html`) será servido como `index.html` principal.

# 6 Limites e requisitos de evolução

Nenhuma das interfaces Web atuais (Flutter Web ou prévia Firebase Hosting) deve ser utilizada com dados pessoais reais ou exposta como serviço produtivo sem a conclusão das salvaguardas técnicas. Para a evolução rumo ao piloto real, estabelecem-se os seguintes requisitos obrigatórios:

1. **Autenticação e controle de acesso:** Substituir os logins simulados por provedores reais (como Firebase Authentication), implementar controle de sessão segura no navegador e configurar políticas de acesso por perfil (PF/PJ) e por tutor.
2. **Persistência e saneamento de dados:** Migrar os registros demonstrativos para banco de dados em nuvem (Firestore ou Supabase) com regras de segurança restritivas, removendo todos os dados fictícios (*seeds*) antes da publicação.

3. **Conformidade com a LGPD:** Implementar fluxos explícitos de consentimento, políticas de retenção e exclusão de dados, controle rigoroso de acesso ao prontuário e exibição da política de privacidade na interface.

4. **Experiência Web e resiliência:** Validar a responsividade em diferentes resoluções, navegação por teclado, compatibilidade com leitores de tela, recarregamento direto de rotas (evitando erros HTTP 404 em SPAs) e tratamento visual de falhas de rede.

5. **Supervisão clínica:** Submeter os fluxos da triagem orientativa à validação de médicos-veterinários parceiros e manter avisos permanentes de não diagnóstico.

# 7 Estrutura relevante

A Tabela 1 consolida os principais arquivos que compõem tanto a aplicação Flutter Web quanto o pacote de prévia no Firebase Hosting.

Tabela 1: Principais arquivos da aplicação Flutter Web e da prévia Firebase Hosting.

| **Arquivo** | **Função** |
|---|---|
| `lib/main.dart` | Ponto de entrada e inicialização da aplicação Flutter. |
| `lib/core/routes/app_router.dart` | Configuração de rotas, parâmetros e redirecionamentos. |
| `lib/core/services/app_data_service.dart` | Gerenciamento de estado em memória da demonstração. |
| `lib/core/services/auth_service.dart` | Fluxo de autenticação simulado sem backend. |
| `web/index.html` e `web/manifest.json` | Shell HTML, metadados e manifesto PWA do Flutter Web. |
| `firebase.json` e `firebaserc` | Configuração do Firebase Hosting e vínculo com o projeto `petcuida-mobile-2026`. |
| `index_1a.html` e `index_0.html` | Prévias estáticas HTML/JS (versão ampliada com PF/PJ e versão inicial). |
| `test/core/services/app_data_service_test.dart` | Suíte de testes unitários da camada de serviços. |

# Referências


[1] GRUPO 4. **Código-fonte compartilhado e shell Web (`flutter/lib/` e `flutter/web/`)**. Belo Horizonte: PUC Minas, 2026.

[2] GRUPO 4. **Pacote de prévia estática Firebase Hosting (`petcuida-mobile-2026`)**. Belo Horizonte: PUC Minas, 2026.

[3] GRUPO 4. **Pacote de entrega `PetCuida_D4C2B_LGPD` e artefatos da Atividade XIV**. Belo Horizonte: PUC Minas, 2026.




**PONTIFÍCIA UNIVERSIDADE CATÓLICA DE MINAS GERAIS**
Campus Coração Eucarístico
Curso de Engenharia de Computação
Disciplina: Tópicos III – Empreendedorismo e Inovação

# Guia de Melhorias Futuras: PetCuida Web e Mobile

Prioridades Técnicas, Segurança, Critérios de Aceite e Roteiro de Evolução

**Integrantes:**

Gabriel da Silva Cassino
Giovanna Naves Ribeiro
Júlia Rodrigues Vasconcellos Melo
Kathleen Rodrigues Leite Barbosa

**Professor-orientador:**
João Carlos Oliveira Caetano
**Professor-tutor:**
Paulo Mattos

Belo Horizonte
2026

**Gabriel da Silva Cassino**
**Giovanna Naves Ribeiro**
**Júlia Rodrigues Vasconcellos Melo**
**Kathleen Rodrigues Leite Barbosa**

# Guia de Melhorias Futuras: PetCuida Web e Mobile

(Plano de Engenharia e Conformidade — Revisão `D4C2B_LGPD`)

Documento técnico apresentado à disciplina Tópicos III – Empreendedorismo e Inovação, do Curso de Engenharia de Computação da PUC Minas, estabelecendo os requisitos técnicos, bloqueadores de segurança e critérios de aceite para a transição do protótipo PetCuida ao ambiente produtivo.

Belo Horizonte
2026

# Sumário

**1 Princípio de planejamento** **2**

**2 Prioridade P0 — Bloqueadores para piloto com dados reais** **2**

**3 Prioridade P1 — Confiabilidade funcional por plataforma** **3**
3.1 Núcleo compartilhado e regras de negócio . . . . . 3
3.2 Plataforma Web . . . . . 3
3.3 Plataforma Android . . . . . 4

**4 Prioridade P2 — Operação, observabilidade e escala** **4**

**5 Critérios de aceite mensuráveis** **4**

**6 Roteiro de execução sugerido** **5**

**Referências** **5**

# 1 Princípio de planejamento

A arquitetura atual do **PetCuida** (`D4C2B_LGPD`) baseia-se em um núcleo Flutter compartilhado pelos destinos Web e Android, complementado por uma camada de serviços e prototipação em Flask. O plano de evolução técnica deve, portanto, consolidar primeiro as regras de domínio, persistência e segurança no núcleo compartilhado, para então validar as particularidades de cada plataforma cliente.

Ressalta-se que a entrega atual opera essencialmente como uma simulação local em memória, não devendo ser confundida com um sistema conectado a banco de dados produtivo, autenticação real ou piloto comercial ativo. As diretrizes apresentadas a seguir distinguem os requisitos bloqueadores de segurança (P0) das melhorias de confiabilidade funcional (P1) e maturidade operacional (P2).

# 2 Prioridade P0 — Bloqueadores para piloto com dados reais

Nenhum dado pessoal de tutores, parceiros ou histórico clínico de animais deve ser coletado em ambiente real antes da conclusão integral dos seguintes requisitos:

1. **Identidade e gestão de sessão:** Substituir o mapa de usuários em memória e o login simulado por um provedor de autenticação persistente (como Firebase Authentication ou Supabase Auth). A sessão deve emitir um identificador único e confiável para o tutor ou organização (PF/PJ), com fluxos definidos para encerramento de sessão (*logout*), expiração de *tokens*, recuperação de senha e estados de carregamento.

2. **Autorização e isolamento entre contas:** Validar rigorosamente a leitura e a escrita entre contas independentes para cada entidade privada. O sistema nunca deve confiar em identificadores de proprietário enviados livremente pelo cliente. Regras de segurança no servidor (*Firestore Security Rules* ou *Supabase Row Level Security*) devem adotar a política de negação por padrão (*default-deny*), separar permissões por operação e contar com testes automatizados positivos e negativos [2, 3].

3. **Modelo de dados e fonte única da verdade:** Definir um único backend principal para produção, versionando migrações de esquema e alinhando serialização e índices. Como catálogo público, perfil privado, organizações PJ, agendamentos, pedidos e movimentações de créditos possuem ciclos de vida distintos, os registros devem ser estruturados em coleções ou tabelas normalizadas, evitando documentos globais monolíticos.

4. **Remoção de dados demonstrativos (*seeds*):** Os registros fictícios devem permanecer restritos ao ambiente local de demonstração. Em ambientes remotos, novas contas devem iniciar vazias ou carregar apenas seus próprios dados, eliminando completamente identificadores compartilhados como `tutor-demo`.

5. **Consistência de interface e tratamento de falhas:** A interface deve confirmar agendamentos, cadastros ou candidaturas somente após o aceite efetivo da operação pelo servidor. É obrigatório exibir indicadores de carregamento, mensagens claras de falha recuperável com opção de nova tentativa e tratamento adequado para perda de conectividade, evitando duplicações ou falsos positivos de sucesso.

6. **Privacidade e segurança operacional (LGPD):** Consolidar os controles previstos no módulo `privacy.py`, inventariando dados coletados, registrando consentimento informado, aplicando minimização de dados (especialmente localização e histórico clínico) e definindo políticas de retenção e exclusão. Chaves administrativas jamais devem ser embutidas no código do cliente, seguindo as diretrizes da ANPD para controle de acesso, menor privilégio e cópias de segurança [4].

# 3 Prioridade P1 — Confiabilidade funcional por plataforma

## 3.1 Núcleo compartilhado e regras de negócio

A camada de apresentação (*widgets*) deve ser desacoplada dos casos de uso e dos SDKs externos por meio de interfaces explícitas para autenticação, perfis (PF e PJ), pets, catálogo, agendamentos, pedidos e carteira de créditos.

O saldo de créditos solidários deve ser calculado exclusivamente a partir de um livro-razão (*ledger*) auditável no servidor, nunca como um campo numérico editável pelo aplicativo cliente. Da mesma forma, o envio de fotos de pets ou documentos exige armazenamento em nuvem autorizado, impedindo que caminhos locais do dispositivo sejam gravados como URLs remotas.

No âmbito clínico, a triagem deve permanecer explicitamente identificada como orientativa. Antes de expandir a árvore de decisões, o fluxo precisa ser revisado por médicos-veterinários parceiros, mantendo alertas destacados para sinais de emergência e validando cenários que possam gerar falsa tranquilização. Transações financeiras reais e conversão de créditos permanecem fora do escopo produtivo até a contratação de provedores regulamentados.

## 3.2 Plataforma Web

- **Responsividade e painéis:** Validar a adaptação visual em telas de dispositivos móveis, *tablets* e computadores, estabelecendo limites adequados de largura de leitura e ergonomia nos painéis do tutor e da pessoa jurídica.

- **Acessibilidade e fluxos críticos:** Realizar inspeção de navegação por teclado, compatibilidade com leitores de tela, contraste de cores, foco visível e tamanho de áreas clicáveis, testando login, cadastro, *onboarding*, formulários e agendamento nos principais navegadores.

- **Metadados e PWA:** Manter consistentes os metadados de idioma (`pt-BR`), título, descrição, *viewport* e manifesto PWA, validando ícones, atualização de versão e comportamento *offline* caso o modo instalável seja adotado.

- **Roteamento e limpeza de sessão:** Testar o recarregamento direto de URLs internas, o ajuste de `base href` em hospedagens sob subdiretórios, o tratamento de páginas não encontradas (erro 404) e a ausência de dados pessoais expostos no console ou armazenamento local do navegador.

### 3.3 Plataforma Android

- **Identidade e assinatura de publicação:** Configurar explicitamente a identidade do pacote, o versionamento e as versões mínima e alvo do SDK. O perfil `release` no Gradle deve ser desvinculado da chave de *debug* e assinado com certificado criptográfico seguro armazenado fora do repositório.
- **Permissões e hardware:** Aplicar o princípio de menor privilégio na solicitação de permissões. Como a câmera é opcional, deve-se testar o comportamento do aplicativo diante da negação ou revogação da permissão, ausência de câmera física e seleção alternativa via galeria.
- **Ciclo de vida e dispositivos:** Validar o comportamento em diferentes densidades de tela, abertura de teclado virtual, retomada após suspensão em segundo plano, perda repentina de rede, botão nativo de voltar e ausência de dados sensíveis nos *logs* do sistema. Use o código com cuidado.
- **Homologação de pacote:** Compilar e instalar o APK ou *App Bundle* de *release* assinado em dispositivos físicos de teste, verificando o fluxo de atualização de versão e os links externos de política de privacidade.

## 4 Prioridade P2 — Operação, observabilidade e escala

Para sustentar a evolução contínua após o piloto inicial, recomenda-se estruturar uma esteira de integração contínua (CI) executando verificação de formatação, análise estática (`flutter analyze`), testes unitários, validação de regras de segurança e geração automatizada dos pacotes Web e Android. A configuração por ambiente (desenvolvimento, homologação e produção) deve ser isolada sem exposição de segredos no cliente. Operacionalmente, é necessário implementar registro de *logs* anonimizados, monitoramento de erros, alertas de falha de sincronização, rotinas testadas de *backup*/restauração e documentação formal de versionamento e reversão (*rollback*). Quanto à escolha definitiva entre Firebase Firestore e Supabase (PostgreSQL), a decisão deve basear-se nos ensaios técnicos dos adaptadores já presentes no repositório, priorizando o Supabase caso a integridade relacional entre entidades (tutores, pets, organizações e transações) se mostre predominante, ou o Firestore caso a sincronização *offline* nativa em tempo real seja o requisito crítico.

## 5 Critérios de aceite mensuráveis

A transição do protótipo para o piloto controlado será considerada apta somente quando todos os critérios abaixo forem verificados:

1. **Integridade transacional:** Cada ação válida gera um único registro no servidor; requisições duplicadas ou identificadores inválidos são rejeitados sem corromper o estado, e falhas de rede nunca são exibidas como sucesso.
2. **Isolamento de privacidade:** Em testes automatizados com duas contas distintas, nenhuma conta consegue ler, alterar ou excluir registros privados da outra, com cobertura explícita de casos permitidos e negados.

3. **Automatização de compilação:** Análise estática, suíte de testes unitários, compilação Web e geração de pacote Android assinado executam com sucesso em ambiente controlado.

4. **Conformidade Web:** Os fluxos essenciais funcionam integralmente em telas móveis e *desktop*, suportam recarregamento direto de rotas sem erro 404 e atendem aos requisitos básicos de acessibilidade e foco.

5. **Conformidade Android:** A instalação e a atualização do pacote de *release* ocorrem sem falhas em aparelhos de teste, a negação de permissões opcionais não bloqueia o aplicativo e não há uso de assinatura de *debug*.

6. **Governança LGPD:** O inventário de dados, termos de consentimento, prazos de retenção, fluxo de exclusão de conta e controle de acesso administrativo estão documentados e validados.

7. **Segurança clínica e financeira:** A linguagem da triagem encontra-se revisada por médico-veterinário e as movimentações de créditos operam via livro-razão no servidor (ou permanecem explicitamente sinalizadas como demonstrativas).

# 6 Roteiro de execução sugerido

A Tabela 1 organiza a sequência recomendada de implementação em cinco fases incrementais, garantindo que a segurança e o isolamento de dados precedam a abertura para usuários reais.

Tabela 1: Fases sequenciais recomendadas para evolução técnica do PetCuida.

| Fase | Atividades principais e entregas |
|---|---|
| **1. Fechamento de escopo** | Validar a cópia atual do código Flutter, congelar os requisitos do piloto assistido e definir o backend produtivo único (Firestore ou Supabase). |
| **2. Identidade e isolamento** | Implementar autenticação real (PF/PJ), modelar entidades separadas, configurar regras *default-deny* e executar testes automatizados de isolamento entre contas. |
| **3. Fluxo vertical básico** | Conectar o primeiro fluxo completo (perfil do tutor e cadastro do pet) à nuvem, validando tratamento de erros, estados de carregamento e remoção de *seeds*. |
| **4. Expansão funcional** | Integrar catálogo de parceiros, agendamentos, painel organizacional e livro-razão de créditos no servidor, submetendo a triagem à revisão veterinária. |
| **5. Homologação e *release*** | Executar testes de responsividade Web, ciclo de vida Android, auditoria de conformidade LGPD e geração dos pacotes assinados para o piloto. |

Até a conclusão integral dessas cinco etapas, o aplicativo deve permanecer estritamente em modo de demonstração controlada, mantendo visível o aviso de dados fictícios em todas as interfaces.

# Referências

[1] GRUPO 4. **Pacote de entrega** $\texttt{PetCuida}_D 4C2B_L GPD$ : *código* − $fonteFlutter, backendFlaskeadaptadores. BeloHorizonte : PUCMinas$, 2026.

[1] GRUPO 4. Propostas técnicas e critérios de aceite para evolução Web e Mobile do PetCuida. Belo Horizonte: PUC Minas, 2026.

[2] FIREBASE. Get started with Cloud Firestore Security Rules. Disponível em: google.com. Acesso em: 11 nov. 2026.

[3] SUPABASE. Row Level Security. Disponível em: supabase.com. Acesso em: 11 nov. 2026.

[4] AUTORIDADE NACIONAL DE PROTEÇÃO DE DADOS (ANPD). Guia orientativo sobre segurança da informação para agentes de tratamento de pequeno porte. Versão 1.0, out. 2021. Disponível em: www.gov.br. Acesso em: 11 nov. 2026.